AETHER KNOWLEDGE

Multi-Agent Knowledge System

LangGraph · RAG · Groq · FAISS · Streamlit
Interview / Portfolio Project Brief

Live demo	https://aetherknowledge.streamlit.app/
GitHub	https://github.com/alirazach476/multi-agent-knowledge-system
Status	Working · Deployed on Streamlit Community Cloud
License	MIT

One-line pitch: A supervisor agent routes each question through a RAG researcher, analyst, and writer over a local knowledge base — with a live Streamlit trace.

1. Project overview

This project demonstrates a **real multi-agent loop** (not a fake linear script). A LangGraph supervisor inspects shared state and decides whether to call the researcher, analyst, writer, or finish. Answers are grounded in Markdown docs under **knowledge/** using FAISS vector search. Chat runs on Groq's free API; embeddings run locally (CUDA GPU when available, CPU on Streamlit Cloud).

Why this is interview-ready

- Shows orchestration skill (LangGraph StateGraph + conditional edges)
- Shows RAG fundamentals (chunking, embeddings, FAISS, citations)
- Shows production-ish habits (env/secrets, fail-fast config, smoke tests, deploy)
- Public live demo + GitHub — easy for interviewers to click and verify

2. Architecture

User question → Supervisor loop → Researcher / Analyst / Writer → Final answer
Specialists return control to the supervisor until **finish** (with a max hop limit to prevent infinite loops).

Agent	Responsibility	Interview talking point
Supervisor	Routes to next specialist via structured output	Shows LangGraph conditional edges + Pydantic schemas
Researcher	FAISS similarity search over knowledge/	Classic RAG retrieval, GPU/CPU embeddings
Analyst	Synthesizes facts and conflicts	Separates reasoning from retrieval
Writer	Final user answer with citations	Grounded generation, citation habit

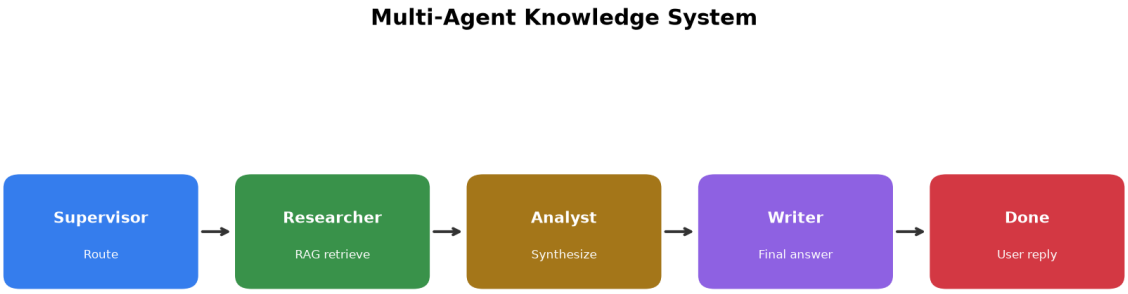


Figure 1 — Pipeline overview (from project outputs)

3. Tech stack

Orchestration	LangGraph (StateGraph, conditional edges, stream updates)
LLM / chat	Groq API — openai/gpt-oss-20b (free tier)
Embeddings	sentence-transformers/all-MiniLM-L6-v2 (local; CUDA FP16 if GPU)

Vector store	FAISS (persisted under vectorstore/, rebuilt on demand)
Validation	Pydantic RouteDecision for structured supervisor routing
UI	Streamlit — live agent trace + branded console (Aether Knowledge)
Config	.env locally · Streamlit Secrets in cloud
Tests	unittest smoke tests (config, graph compile, corpus)

4. Repository layout

```
app.py            Streamlit UI (live trace)
multi_agent.py     Agents, RAG, LangGraph wiring
knowledge/         5 Markdown source docs
tests/            Smoke tests
outputs/          Screenshots + this PDF
.streamlit/        Theme + secrets example
requirements.txt  Pinned dependencies
```

Knowledge corpus (demo internal docs)

- onboarding_faq.md — API keys, models, audit logs
- platform_notes.md — retrieval strategy, evaluation, guardrails
- incident_runbook.md — SEV levels and escalation
- data_retention.md — transcripts, embeddings, training use
- support_sla.md — response targets and channels

5. End-to-end flow (what happens on one question)

- Supervisor reads state flags (notes / analysis / answer empty?) and emits RouteDecision.
- Researcher embeds the query (GPU/CPU), retrieves top-k FAISS passages, writes research_notes.
- Analyst summarizes facts and conflicts from notes.
- Writer drafts a concise final answer and cites document names when possible.
- UI streams each node update so the interviewer sees the team collaborate live.

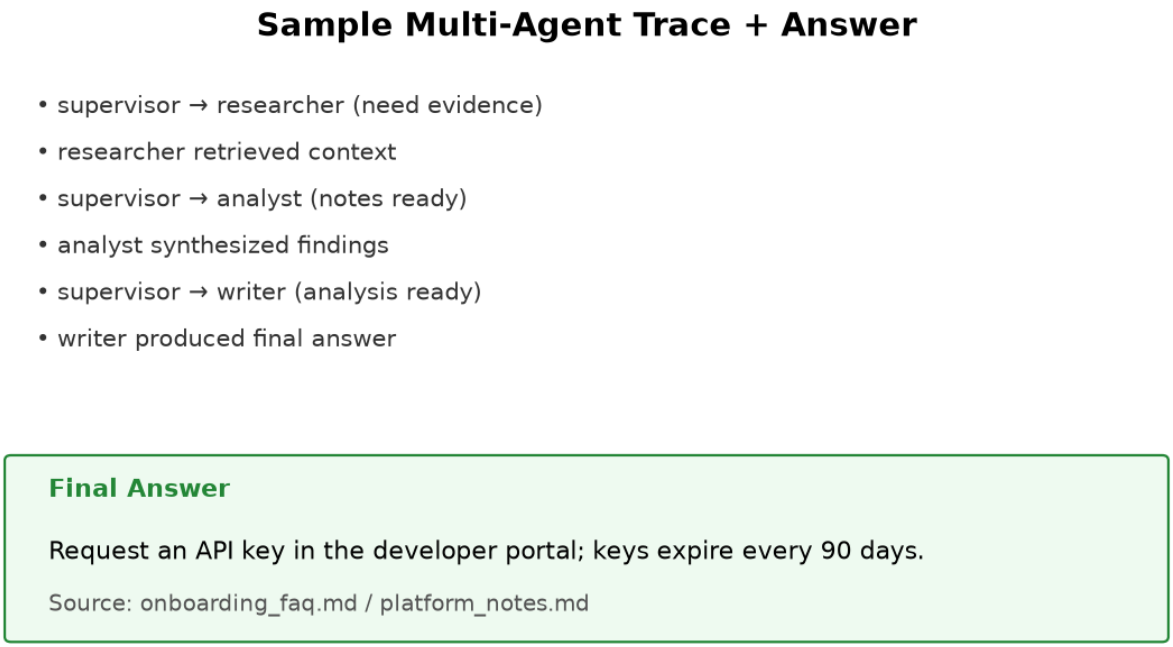


Figure 2 — Sample multi-agent trace and answer

6. How to run / verify (for interviewers)

Fastest proof it works: open the live demo.

Live: https://aetherknowledge.streamlit.app/

Local run

```
python -m venv .venv
.venv\Scripts\activate  # Windows
pip install -r requirements.txt
copy .env.example .env  # set GROQ_API_KEY
streamlit run app.py
python -m unittest discover -s tests -v
```

Example questions to demo

- How do I get API access and how often do keys expire?
- What retrieval strategy and chunk sizes do we use?
- Where are audit logs and how long are they retained?
- What is the on-call escalation path for a production RAG outage?

7. Design decisions & trade-offs (say this in interviews)

- **Supervisor loop vs linear chain:** looping mirrors real agent teams and can revisit research if needed.
- **Groq instead of paid OpenAI:** free chat API; embeddings stay local so RAG still works offline from LLM vendor.
- **Structured routing (Pydantic):** reduces free-text routing errors vs parsing natural language.
- **Hop / recursion limits:** guards against infinite supervisor cycles.
- **GPU optional:** CUDA FP16 locally for speed; auto CPU on Streamlit Cloud so deploy stays portable.
- **Small corpus by design:** clear demo of grounding — not a claim of enterprise search scale.

8. What I would improve next

- Hybrid retrieval (keyword + dense) and metadata filters
- Evaluation harness (faithfulness / relevancy on a golden set)
- Streaming token UI for the writer agent
- Auth + per-tenant knowledge bases for a production SaaS shape
- CI: lint + smoke tests on every push

Prepared as an interview portfolio brief · Project is live and working · Source: github.com/alirazach476/multi-agent-knowledge-system